

Unit – 1 [Part 1]

Object Oriented Programming in PHP

☺ The Basics:

Object-Oriented Programming (OOP) is a type of programming added to php5 that makes building complex, modular and reusable web applications that much easier. With the release of php5, php programmers finally had the power to code with the 'big boys'. Like Java and C#, php finally has a complete OOP infrastructure. Object Oriented Concepts important terms related to Object Oriented Programming:-

- **Class** – This is a programmer-defined data type, which includes local functions as well as local data. You can think of a class as a template for making many instances of the same kind (or class) of object.
- **Object** – An individual instance of the data structure defined by a class. You define a class once and then make many objects that belong to it. Objects are also known as instance.
- **Member Variable** – These are the variables defined inside a class. This data will be invisible to the outside of the class and can be accessed via member functions. These variables are called attribute of the object once an object is created.
- **Member function** – These are the function defined inside a class and are used to access object data.
- **Inheritance** – When a class is defined by inheriting existing function of a parent class then it is called inheritance. Here child class will inherit all or few member functions and variables of a parent class.
- **Parent class** – A class that is inherited from by another class. This is also called a base class or super class.
- **Child Class** – A class that inherits from another class. This is also called a subclass or derived class.
- **Polymorphism** – This is an object oriented concept where same function can be used for different purposes. For example function name will remain same but it make take different number of arguments and can do different task.
- **Overloading** – a type of polymorphism in which some or all of operators have different implementations depending on the types of their arguments. Similarly functions can also be overloaded with different implementation.
- **Data Abstraction** – Any representation of data in which the implementation details are hidden(abstracted).
- **Encapsulation** – refers to a concept where we encapsulate all the data and member functions together to form an object.
- **Constructor** – refers to a special type of function which will be called automatically whenever there is an object formation from a class.
- **Destructor** – refers to a special type of function which will be called automatically whenever an object is deleted or goes out of scope.

☺ CLASS

Basic class definitions begin with the keyword *class*, followed by a class name, followed by a pair of curly braces which enclose the definitions of the properties and methods belonging to the class.

A class is a template for a real world object to be used in a program.

The class name can be any valid label, provided it is not a PHP reserved word.

A valid class name starts with a letter or underscore, followed by any number of letters, numbers, or underscores.

For example,

- A programmer can create a car class which emulates a car.
- This class can contain the properties of a car (color, model, year, etc.) and methods (functions) that specify what the car does (drive, reverse, stop, etc.).

A class may contain its own constants, variables (called "properties"), and functions (called "methods").

Syntax:

```
class className
{
    .....
}
```

Example #1 Simple Class definition

```
<?php
class SimpleClass
{
    // property declaration
    public $var = 'a default value';

    // method declaration
    public function
        displayVar()
    {
        echo $this->var;
    }
}
$a = new SimpleClass();

$a->displayVar();
?>
```

The pseudo-variable *\$this* is available when a method is called from within an object context. *\$this* is a reference to the calling object.

Example #2 *\$this* pseudo-variable

We're assuming that `error_reporting` is disabled for this example; otherwise the following code would trigger deprecated and strict notices, respectively, depending on the PHP version.

```
<?php
class A
{
    function foo()
    {
        if (isset($this))
        {
            echo    '$this is defined';
            echo    get_class($this);
            echo    "<br>";
        }
        else
        {
            echo    "\$this is not defined.<br>";
        }
    }
}

class B
{
    function bar()
    {
        A::foo();
    }
}
```

```

$a = new A();
$a->foo();

A::foo();
$b = new B();
$b->bar();

B::bar();
?>

```

Output of the above example in PHP 5:

```

$this is defined (A)
$this is not defined.
$this is defined (B)
$this is not defined.

```

Output of the above example in PHP 7:

```

$this is defined (A)
$this is not defined.
$this is not defined.
$this is not defined.

```

- **Creating variables for a class**

Class variables are known as properties, because they store important information in regards to the class - hence, they are the classes properties. We will be referring to class variables as 'variables' or 'properties' interchangeably throughout this lesson.

Example:

```

class Book
{
    var $title;
    var $numPages;
}

```

- **Instantiating a class or creating object:**

So you have created your class with its variables and functions, and now what to do with it? Now you can create objects based on how you designed your class. The process of creating an object is called **instantiation**. You can create an object by creating a variable which stores the object, and using the **new** keyword and the name of the class to instantiate from.

Syntax: \$objectName = new nameOfClass();

Example: \$yellowPages = new Book();

- **NEW**

To create an instance of a class, the *new* keyword must be used. An object will always be created unless the object has a constructor defined that throws an exception on error. Classes should be defined before instantiation (and in some cases this is a requirement).

If a string containing the name of a class is used with *new*, a new instance of that class will be created. If the class is in a namespace, its fully qualified name must be used when doing this.

Example #3 Creating an instance

```
<?php
$instance = new SimpleClass();

// This can also be done with a variable:

$className = 'SimpleClass';

$instance = new $className(); // new
SimpleClass()

?>
```

In the class context, it is possible to create a new object by *new self* and *new parent*.

When assigning an already created instance of a class to a new variable, the new variable will access the same instance as the object that was assigned. This behaviour is the same when passing instances to a function. A copy of an already created object can be made by cloning it.

Example #4 Object Assignment

```
<?php

$instance = new SimpleClass();

$assigned      =
$instance;
$reference      =&
$instance;

$instance->var = '$assigned will have this value';

$instance = null; // $instance and $reference become null

var_dump($instance)
;
var_dump($reference
);
var_dump($assigned)
;
?>
```

The above example will output:

```
NULL
NULL

object(SimpleClass)#
  1 (1) { ["var"]=>
    string(30) "$assigned will have this value"
  }
```

PHP 5.3.0 introduced a couple of new ways to create instances of an object:

Example #5 Creating new objects

```
<?ph
p
clas
```

```

s
Test
{
    static public function getNew()
    {
        return new static;
    }
}

class      Child
extends Test {

}

$obj1 = new
Test(); $obj2
= new $obj1;
var_dump($obj1 !== $obj2);

$obj3 = Test::getNew();
var_dump($obj3 instanceof
Test);

$obj4 = Child::getNew();
var_dump($obj4 instanceof
Child); ?>

```

The above example will output:

```

bool(true)
bool(true)
bool(true)

```

• **PROPERTIES AND METHODS**

Class properties and methods live in separate "namespaces", so it is possible to have a property and a method with the same name. Referring to both a property and a method has the same notation, and whether a property will be accessed or a method will be called, solely depends on the context, i.e. whether the usage is a variable access or a function call.

Example #6 Property access vs. method call

```

class Foo
{
    public $bar = 'property';

    public function
        bar()      {
            return
                'method';
        }
}

$obj = new Foo();
echo $obj->bar, PHP_EOL, $obj->bar();

```

The above example will output:

```

property
method

```

😊 Properties:

Class member variables are called "properties". You may also see them referred to using other terms such as "attributes" or "fields", but for the purposes of this reference we will use "properties". They are defined by using one of the keywords *public*, *protected*, or *private*, followed by a normal variable declaration. This declaration may include an initialization, but this initialization must be a constant value--that is, it must be able to be evaluated at compile time and must not depend on run-time information in order to be evaluated.

Note:

In order to maintain backward compatibility with PHP 4, PHP 5 will still accept the use of the keyword *var* in property declarations instead of *public*, *protected*, or *private*. However, *var* is no longer required. In versions of PHP from 5.0 to 5.1.3, the use of *var* was considered deprecated and would issue an `E_DEPRECATED` warning, but since PHP 5.1.3 it is no longer deprecated and does not issue the warning.

If you declare a property using *var* instead of one of *public*, *protected*, or *private*, then PHP 5 will treat the property as if it had been declared as *public*.

Example #1 property declarations

```
<?php
class SimpleClass
{
    public $var1 = 'hello ' . 'world';

    public $var2 = 1+2;

}
?>
```

Example #2 property declarations

```
<?php
class SimpleClass
{
    // valid as of PHP 5.6.0:
    public $var1 = 'hello ' . 'world';

    // valid as of PHP 5.3.0:
    public $var2 = <<<EOD
    hello
    world
    EOD;

    // valid as of PHP 5.6.0:
    public $var3 = 1+2;

    // invalid property declarations:
    public $var4 = self::myStaticMethod();
    public $var5 = $myVar;
    // valid property declarations:
    public $var6 = myConstant;
    public $var7 = array(true, false);
    // valid as of PHP 5.3.0:
    public $var8 = <<<'EOD'
    hello
    world
    EOD;

}
?>
```

☺ Class constants:

It is possible to define constant values on a per-class basis remaining the same and unchangeable. Constants differ from normal variables in that you don't use the \$ symbol to declare or use them.

The value must be a constant expression, not (for example) a variable, a property, or a function call.

It's also possible for interfaces to have *constants*. Look at the [interface documentation](#) for examples.

Example #1 Defining and using a constant

```
<?php
class MyClass
{
    const CONSTANT = 'constant value';

    function showConstant() {
        echo self::CONSTANT . "\n";
    }
}

echo MyClass::CONSTANT . "\n";

$classname = "MyClass";
echo $classname::CONSTANT . "\n"; // As of PHP 5.3.0

$class = new MyClass();
$class->showConstant();

echo $class::CONSTANT . "\n"; // As of PHP 5.3.0
?>
```

☺ Auto loading Classes:

Many developers writing object-oriented applications create one PHP source file per class definition. One of the biggest annoyances is having to write a long list of needed includes at the beginning of each script (one for each class).

In PHP 5, this is no longer necessary. The [spl_autoload_register\(\)](#) function registers any number of autoloaders, enabling for classes and interfaces to be automatically loaded if they are currently not defined. By registering autoloaders, PHP is given a last chance to load the class or interface before it fails with an error.

Whilst the `__autoload()` function can also be used for autoloading classes and interfaces

Example #1 Autoload example

This example attempts to load the classes *MyClass1* and *MyClass2* from the files *MyClass1.php* and *MyClass2.php* respectively.

```
<?php
spl_autoload_register(function ($class_name) {
    include $class_name . '.php';
});
```

```
$obj = new MyClass1();
$obj2= new MyClass2();
?>
```

Example #2 Autoload other example

```
<?php

spl_autoload_register(function ($name) {
    var_dump($name);
});

class Foo implements ITest {
}

/*
string(5) "ITest"

Fatal error: Interface 'ITest' not found in ...
*/
?>
```

☺ Constructors and Destructors:

CONSTRUCTOR

PHP 5 allows developers to declare constructor methods for classes. Classes which have a constructor method call this method on each newly-created object, so it is suitable for any initialization that the object may need before it is used.

If a class name and function name will be similar in that case function is known as constructor. Constructor is special type of method because its name is similar to class name. Constructor automatically calls when object will be initializing.

PHP provides a special function called **__construct()** to define a constructor. You can pass as many as arguments you like into the constructor function.

```
void __construct ([ mixed $args = "" [, $... ] ] )
```

1 - Syntax For PHP 4.0

Class Demo

```
{
    function Demo()
    {
        //Statement
    }
}
```

2 - Syntax For PHP 5.0

Class Demo

```
{
    function __construct
    {
        //Statment
    }
}
```



```
}
```

Example #1 using new unified constructors

```
<?php

classBaseClass {
    function __construct() {
        print "In BaseClass constructor\n";
    }
}

classSubClass extends BaseClass {
    function __construct() {
        parent::__construct();
        print "In SubClass constructor\n";
    }
}

classOtherSubClass extends BaseClass
{
    // inherits BaseClass's constructor
}

// In BaseClass constructor
$obj = new BaseClass();

// In BaseClass constructor
// In SubClass constructor

$obj = new SubClass();

// In BaseClass constructor

$obj = new OtherSubClass();

?>
```

Example #2 Constructors in namespaced classes

```
<?php
namespace Foo;
class Bar {
    public function Bar() {
        // treated as constructor in PHP 5.3.0-5.3.2
        // treated as regular method as of PHP 5.3.3
    }
}
?>
```

DESTRUCTOR

void **__destruct** (void)

PHP 5 introduces a destructor concept similar to that of other object-oriented languages, such as C++. The destructor method will be called as soon as there are no other references to a particular object, or in any order during the shutdown sequence or when the object is explicitly destroyed.

Like a constructor function you can define a destructor function using function **__destruct()**. You can release all the resources with-in a destructor.

Syntax :

Class Demo

```
{
Function __destruct()
{
//Statment
}
}
```

Example : Destructor Example

```
<?php
classMyDestructableClass {
    function __construct() {
        print "In constructor\n";
        $this->name ="MyDestructableClass";
    }

    function __destruct() {
        print "Destroying " . $this->name . "\n";
    }
}

$obj = new
MyDestructableClass(); ?>
```

😊 Visibility:

The visibility of a property or method can be defined by prefixing the declaration with the keywords *public*, *protected* or *private*.

There are 3 type of visibility available in php for controlling your property or method.

1. **Public:**

Class members declared public can be accessed everywhere.

scope to make that variable/function available from anywhere, other classes and instances of the object.

Public method or variable can be accessible from anywhere. I mean from inside the class, outside the class and in child class also.

2. **Private:**

Members declared as private may only be accessed by the class that defines the member.

Scope when you want your variable/function to be visible in its own class only

Method or property with private visibility can only be accessible inside the class. You cannot access private method or variable from outside of your class.

3. **Protected:**

Members declared protected can be accessed only within the class itself and by inherited classes

scope when you want to make your variable/function visible in all classes that extend current class including the parent class.

Method or variable with protected visibility can only be access in the derived class. Or in other word in child class. Protected will be used in the process of inheritance

Class member access specifier	Access from own Class	Accessible from derived class	Access from object
Private member	Yes	No	No
Protected member	Yes	Yes	No
Public member	Yes	Yes	Yes

```
<?php
```

```
class MyClass
{
    public $public = 'Public';
    protected $protected = 'Protected';
    private $private = 'Private';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj = new MyClass();

echo $obj->public; // Works
//echo $obj->protected; // Fatal Error
//echo $obj->private; //Fatal Error
$obj->printHello();
// Shows Public, Protected and Private
?>
```

PROPERTY VISIBILITY

Class properties must be defined as public, private, or protected. If declared using *var*, the property will be defined as public.

Example #1 Property declaration

```
<?php
Class MyClass
{
    public $public = 'Public'; protected
    $protected = 'Protected'; private
    $private = 'Private';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj = new MyClass();
echo $obj->public; // Works
echo $obj->protected; // Fatal Error
echo $obj->private; // Fatal Error
```

```
$obj->printHello(); // Shows Public, Protected and Private
```

```
class MyClass2 extends MyClass
{
    // We can redeclare the public and protected method, but not private
    protected $protected = 'Protected2';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj2 = new MyClass2();
echo $obj2->public; // Works
echo $obj2->protected; // Fatal Error
echo $obj2->private; // Undefined
$obj2->printHello(); // Shows Public, Protected2,

Undefined

?>
```

METHOD VISIBILITY

Class methods may be defined as public, private, or protected. Methods declared without any explicit visibility keyword are defined as public.

Example #1 Method Declaration

```
<?php
/**
 * Define MyClass */

class MyClass
{
    // Declare a public constructor
    public function __construct() { }

    // Declare a public method
    public function MyPublic()
    {
        echo 'hello';
    }

    // Declare a protected method
    protected function MyProtected()
    {
        echo 'hello';
    }

    // Declare a private method
    private function MyPrivate() {
    }

    public function Foo()
    {

        $this->MyPublic();
        $this->MyProtected();
    }
}
```

```

    $this->MyPrivate();
}

}

class MyClass2 extends MyClass

{
public function Foo2()

{

$this->MyPublic();
$this->MyProtected();
// $this->MyPrivate(); // Fatal Error

}

}

$myclass2 = new MyClass2;
$myclass2->MyPublic(); // Works

$myclass2->Foo2(); // Public and Protected work, not Private

?>

```

VISIBILITY FROM OTHER OBJECTS

Objects of the same type will have access to each others private and protected members even though they are not the same instances. This is because the implementation specific details are already known when inside those objects.

Example #3 Accessing private members of the same object type

```

<?php
class Test
{
    private $foo;

    public function __construct($foo)
    {
        $this->foo = $foo;
    }

    private function bar()
    {
        echo 'Accessed the private method.';
    }

    public function baz(Test $other)
    {
        // We can change the private property:
        $other->foo = 'hello';
        // We can also call the private method:
        $other->bar();
    }
}

$test = new Test('test');

$test->baz(new Test('other'));

?>

```

The above example will output:

```
string(5) "hello"  
Accessed the private method.
```

☺ Inheritance in PHP :

Inheritance is a concept in object oriented programming. With the help of inheritance we can get all property and method of one class in another class.

What is inheritance?

Inheritance is nothing but a design principle in oop. By implementing inheritance you can inherit(or get) all properties and methods of one class to another class. The class who inherit feature of another class known as **child class**. The class which is being inherited is known as **parent class**. With the help of inheritance you can increase re-usability of code.

Syntax:

```
class Parent  
{  
    // The parent's class code  
}  
class Child extends Parent  
{  
    // The child can use the parent's class code  
}
```

☺ Object Inheritance:

Inheritance is a well-established programming principle, and PHP makes use of this principle in its object model. This principle will affect the way many classes and objects relate to one another.

For example, when you extend a class, the subclass inherits all of the public and protected methods from the parent class. Unless a class overrides those methods, they will retain their original functionality.

This is useful for defining and abstracting functionality, and permits the implementation of additional functionality in similar objects without the need to reimplement all of the shared functionality.

Note:

Unless autoloading is used, then classes must be defined before they are used. If a class extends another, then the parent class must be declared before the child class structure. This rule applies to classes that inherit other classes and interfaces.

PHP class definitions can optionally inherit from a parent class definition by using the extends clause. The syntax is as follows –

```
class Child extends Parent  
{  
<definition body>  
}
```

The effect of inheritance is that the child class (or subclass or derived class) has the **following characteristics –**

- Automatically has all the member variable declarations of the parent class.
- Automatically has all the same member functions as the parent, which (by default) will work the same way as those functions do in the parent.

Example #1 Inheritance Example

```
<?php

class Foo
{
    public function printItem($string)
    {
        echo 'Foo: ' . $string;
    }

    public function printPHP()
    {
        echo 'PHP is great.' . PHP_EOL;
    }
}

class Bar extends Foo
{
    public function printItem($string)
    {
        echo 'Bar: ' . $string . PHP_EOL;
    }
}

$foo = new Foo();
$bar = new Bar();
$foo->printItem('baz'); // Output: 'Foo: baz'
$foo->printPHP();      // Output: 'PHP is great'
$bar->printItem('baz'); // Output: 'Bar: baz'
$bar->printPHP();      // Output: 'PHP is great'

?>
```

☺ Scope Resolution Operator (::)

The Scope Resolution Operator (also called Paamayim Nekudotayim) or in simpler terms, the double colon, is a token that allows access to static, constant, and overridden properties or methods of a class.

When referencing these items from outside the class definition, use the name of the class.

As of PHP 5.3.0, it's possible to reference the class using a variable. The variable's value can not be a keyword (e.g. *self*, *parent* and *static*).

Example #1 :: from outside the class definition

```
<?php
class MyClass {
    const CONST_VALUE = 'A constant value';
}
```

```

$classname = 'MyClass';
echo $classname::CONST_VALUE; // As of PHP 5.3.0

echo MyClass::CONST_VALUE;
?>

```

Three special keywords *self*, *parent* and *static* are used to access properties or methods from inside the class definition.

Example #2 :: from inside the class definition

```

<?php
class OtherClass extends MyClass
{
    public static $my_static = 'static var';

    public static function doubleColon() {
        echo parent::CONST_VALUE . "\n";
        echo self::$my_static . "\n";
    }
}

$classname = 'OtherClass';
echo $classname::doubleColon(); // As of PHP 5.3.0

OtherClass::doubleColon();
Otherclass->
?>

```

When an extending class overrides the parents definition of a method, PHP will not call the parent's method. It's up to the extended class on whether or not the parent's method is called. This also applies to Constructors and Destructors, Overloading, and Magic method definitions.

Example #3 Calling a parent's method

```

<?php
class MyClass
{
    protected function myFunc() {
        echo "MyClass::myFunc() \n";
    }
}

class OtherClass extends MyClass
{
    // Override parent's definition
    public function myFunc()
    {
        // But still call the parent function
        parent::myFunc();
        echo "OtherClass::myFunc() \n";
    }
}

$class = new OtherClass();
$class::myFunc();
?>

```


😊 Static Keyword:

Static keyword is used to define static methods and properties. *static* can also be used to define static variables and for late static bindings.

Declaring class properties or methods as static makes them accessible without needing an instantiation of the class. A property declared as static cannot be accessed with an instantiated class object (though a static method can).

For compatibility with PHP 4, if no visibility declaration is used, then the property or method will be treated as if it was declared as *public*.

STATIC METHODS

Because static methods are callable without an instance of the object created, the pseudo-variable *\$this* is not available inside the method declared as static.

Caution

In PHP 5, calling non-static methods statically generates an **E_STRICT** level warning.

Warning

In PHP 7, calling non-static methods statically is deprecated, and will generate an **E_DEPRECATED** warning. Support for calling non-static methods statically may be removed in the future.

Example #1 Static method example

```
<?php
class Foo {
    public static function aStaticMethod()
    {
        echo "hello";
    }
}

Foo::aStaticMethod();
$classname = 'Foo';
$classname::aStaticMethod(); //As of PHP5.3.0
?>
```

STATIC PROPERTIES

Static properties cannot be accessed through the object using the arrow operator ->.

Like any other PHP static variable, static properties may only be initialized using a literal or constant before PHP 5.6; expressions are not allowed. In PHP 5.6 and later, the same rules apply as const expressions: some limited expressions are possible, provided they can be evaluated at compile time.

As of PHP 5.3.0, it's possible to reference the class using a variable. The variable's value cannot be a keyword (e.g. *self*, *parent* and *static*).

Example #2 Static property example

```
<?php
class Foo
{
```

```

    public static $my_static = 'foo';

    public function staticValue() {
        return self::$my_static;
    }
}

class Bar extends Foo
{
    public function fooStatic() {
        return parent::$my_static;
    }
}

print Foo::$my_static . "\n";

$foo = new Foo();
print $foo->staticValue() . "\n";
print $foo->my_static . "\n";          // Undefined "Property" my_static

print $foo::$my_static . "\n";
$classname = 'Foo';
print $classname::$my_static . "\n"; // As of PHP 5.3.0

print Bar::$my_static . "\n";
$bar = new Bar();
print $bar->fooStatic() . "\n";
?>

```

☺ Class Abstraction:

PHP 5 introduces abstract classes and methods. Classes defined as abstract may not be instantiated, and any class that contains at least one abstract method must also be abstract. Methods defined as abstract simply declare the method's signature - they cannot define the implementation.

Abstract method are the method which is only defined but declared. This is not true definition as per my assumption. But your any class has at least one method abstract than your class is abstract class.

Usually abstract class are also known as base class. We call it base class because abstract class are not the class which is available directly for creating object.

It can only act as parent class of any normal class. You can use abstract class in class hierarchy. Mean one abstract class can inherit another abstract class also.

Abstract classes are special because they can never be instantiated. Instead, you typically inherit a set of base functionality from them in a new class. For that reason, they are commonly used as the base classes in a larger class hierarchy.

When inheriting from an abstract class, all methods marked abstract in the parent's class declaration must be defined by the child; additionally, these methods must be defined with the same (or a less restricted) visibility. For example, if the abstract method is defined as protected, the function implementation must be defined as either protected or public, but not private. Furthermore the signatures of the methods must match, i.e. the type hints and the number of required arguments must be the same. For example, if the child class defines an optional argument, where the abstract method's signature does not, there is no conflict in the

signature. This also applies to constructors as of PHP 5.4. Before 5.4 constructor signatures could differ.

Example #1 Abstract class example

```
<?php
abstract class AbstractClass
{
    // Force Extending class to define this method
    abstract protected function getValue();
    abstract protected function prefixValue($prefix);

    // Common method
    public function printOut() {
        print $this->getValue() . "\n";
    }
}

class ConcreteClass1 extends AbstractClass
{
    protected function getValue() {
        return "ConcreteClass1";
    }

    public function prefixValue($prefix) {
        return "{$prefix}ConcreteClass1";
    }
}

class ConcreteClass2 extends AbstractClass
{
    public function getValue() {
        return "ConcreteClass2";
    }

    public function prefixValue($prefix) {
        return "{$prefix}ConcreteClass2";
    }
}

$class1 = new ConcreteClass1;
$class1->printOut();
echo $class1->prefixValue('FOO_') . "\n";

$class2 = new ConcreteClass2;
$class2->printOut();
echo $class2->prefixValue('FOO_')
. "\n"; ?>
```

The above example will output:

```
ConcreteClass1
FOO_ConcreteClass1
ConcreteClass2
FOO_ConcreteClass2
```

Example #2 Abstract class example

```
<?php
abstract class AbstractClass
```

```

{
// Our abstract method only needs to define the required arguments
abstract protected function prefixName($name);
}

class ConcreteClass extends AbstractClass
{
    // Our child class may define optional arguments not in the parent's
    signature
    public function prefixName($name, $separator = ".")
    {
        if ($name == "Pacman") {
            $prefix = "Mr";
        } elseif ($name == "Pacwoman")
        {
            $prefix = "Mrs";
        } else {
            $prefix = "";
        }
        return "{$prefix}{$separator} {$name}";
    }
}

$class = new ConcreteClass;
echo $class->prefixName("Pacman"), "\n";
echo $class->prefixName("Pacwoman"),
"\n"; ?>

```

The above example will output:

```

Mr. Pacman
Mrs. Pacwoman

```

Old code that has no user-defined classes or functions named 'abstract' should run without modifications.

☺ Object Interface:

Object interfaces allow you to create code which specifies which methods a class must implement, without having to define how these methods are handled.

Interfaces are defined in the same way as a class, but with the *interface* keyword replacing the *class* keyword and without any of the methods having their contents defined.

All methods declared in an interface must be public; this is the nature of an interface.

Interface in oop enforces definition of some set of method in the class.

By implementing interface you are forcing any class to must declare some specific set of methods in oop.

Interface in php can be implemented like other oop language. You can create interface in php using keyword `interface`. By implementation of interface in php class you are specifying set of the method which classes must implement.

You can create interface in php using `interface` keyword. Rest of the things are typically identical to classes. Following is very small example of interface in php.

```
interface abc
{
    public function xyz($b);
}
```

To implement an interface, the *implements* operator is used. All methods in the interface must be implemented within a class; failure to do so will result in a fatal error. Classes may implement more than one interface if desired by separating each interface with a comma.

```
class test implements abc
{
    public function xyz($b)
    {
        //your function body
    }
}
```

Interfaces can be extended like classes using the extends operator.

Examples

Example #1 Interface example

```
<?php

//Declare the interface two method is they setVariable,getHtml

<?php
interface iTemplate
{

    public function setVariable($name, $var);
    public function getHtml($template);

}
class Template implements iTemplate
{

    private $vars = array();

    public function setVariable($name, $var)
    {

        $this->vars[$name] = $var;
        echo $var;

    }
    public function vip()
    {

        echo "heloo";

    }

    public function getHtml($template)
    {

        echo $template;

    }

}
$template = new Template();
$template->vip();
echo "<br>";
$template->setVariable('vipul','ok');

?>
```

Example #2 Extendable Interfaces

```
<?php

interface a
{
    public function foo();
}

interface b extends a
{
    public function baz(Baz $baz);
}

// This will work
class c implements b
{
    public function foo()
    {
    }

    public function baz(Baz $baz)
    {
    }
}

// This will not work and result in a fatal error
class d implements b
{
    public function foo()
    {
    }

    public function baz(Foo $foo)
    {
    }
}
?>
```

Example #3 Multiple interface inheritance

```
<?php
interface a
{
    public function foo();
}

interface b
{
    public function bar();
}

interface c extends a, b
{
    public function baz();
}

class d implements c
{
    public function foo()
    {
    }
}
```

```

        public function bar()
        {
        }

        public function baz()
        {
        }
    }
    ?>

```

☺ Anonymous Classes:

Support for anonymous classes was added in PHP 7. Anonymous classes are useful when simple, one-off objects need to be created.

```

<?php

// Pre PHP 7 code
class Logger
{
    public function log($msg)
    {
        echo $msg;
    }
}

$util->setLogger(new Logger());

// PHP 7+ code
$util->setLogger(new class {
    public function log($msg)
    {
        echo $msg;
    }
});

```

They can pass arguments through to their constructors, extend other classes, implement interfaces, and use traits just like a normal class can:

```

<?php

classSomeClass {}
interface SomeInterface{}
trait SomeTrait {}

var_dump(new class(10) extends SomeClass implements SomeInterface {
    private $num;

    public function __construct($num)
    {
        $this->num = $num;
    }

    useSomeTrait;
});

```

The above example will output:

```

object(class@anonymous)#1 (1) {
    ["Command line
    code0x104c5b612": "class@anonymous":private]=>int(10)

```

```
}
```

Another example:

```
<?php
```

```
interface Logger
```

```
{
```

```
public function log(string $msg)
```

```
}
```

```
class Final
```

```
{
```

```
    private $logger;
```

```
    public function getLogger():Logger{
```

```
        return $this->logger;
```

```
    }
```

```
    public function setLogger(Logger $logger)
```

```
    {
```

```
        $this->logger=$logger;
```

```
    }
```

```
}
```

```
$app=new Final;
```

```
$app->setLogger(new class implements Logger{
```

```
    public function log(string $msg)
```

```
    {
```

```
        print($msg);
```

```
    }
```

```
});
```

```
$app->getLogger()->log("hi developer");
```

```
?>
```

☺ Overloading:

Overloading in PHP provides means to dynamically "create" properties and methods. These dynamic entities are processed via magic methods one can establish in a class for various action types.

The overloading methods are invoked when interacting with properties or methods that have not been declared or are not visible in the current scope. Also use the terms "inaccessible properties" and "inaccessible methods" to refer to this combination of declaration and visibility.

All overloading methods must be defined as *public*.

Note:

None of the arguments of these magic methods can be passed by reference. PHP's interpretation of "overloading" is different than most object oriented languages. Overloading traditionally provides the ability to have multiple methods with the same name but different quantities and types of arguments.

PROPERTY OVERLOADING

```
public void __set ( string $name , mixed $value )
```

```
public mixed __get ( string $name )
```

```
public bool __isset ( string $name )
```

```
public void __unset ( string $name )
```

__set() is run when writing data to inaccessible properties.

__get() is utilized for reading data from inaccessible properties.

__isset() is triggered by calling isset() or empty() on inaccessible properties.

__unset() is invoked when unset() is used on inaccessible properties.

The *\$name* argument is the name of the property being interacted with. The __set() method's *\$value* argument specifies the value the *\$name*'ed property should be set to.

Property overloading only works in object context. These magic methods will not be triggered in static context. Therefore these methods should not be declared static. As of PHP 5.3.0, a warning is issued if one of the magic overloading methods is declared *static*.

Example #1 Overloading properties via the __get(), __set(), __isset() and __unset() methods

```
<?php
classPropertyTest
{
    private $data = array();

    public $declared = 1;

    private $hidden = 2;

    public function __set($name, $value)
    {
        echo "Setting '$name' to '$value'\n";
        $this->data[$name] = $value;
    }

    public function __get($name)
    {
        echo "Getting '$name'\n";
        if (array_key_exists($name, $this->data))
        {
            return $this->data[$name];
        }
    }

    function __isset($name)
```

```

    {
        echo "Is '$name' set?\n";
        return isset($this->data[$name]);
    }

    function __unset($name)
    {
        echo "Unsetting '$name'\n";
        unset($this->data[$name]);
    }

    /** Not a magic method, just here for example. */
    public function getHidden()
    {
        return $this->hidden;
    }
}
echo "<pre>\n";

$obj = new PropertyTest;

$obj->a = 1;
echo $obj->a . "\n\n";

var_dump(isset($obj->a));
unset($obj->a);
var_dump(isset($obj->a));
echo "\n";

echo $obj->declared . "\n\n";

echo "Privates are visible inside the class, so __get() not used...\n";
echo $obj->getHidden() . "\n";
?>

```

The above example will output:

```

Setting 'a' to '1'
Getting 'a'
1

Is 'a' set?
bool(true)
Unsetting 'a'
Is 'a' set?
bool(false)

1
Privates are visible inside the class, so __get() not used...
2

```

Notice: Undefined property via __get(): hidden in <file> on line 70 in <file> on line 29

METHOD OVERLOADING

public mixed __call (string \$name , array \$arguments)

public static mixed __callStatic (string \$name , array \$arguments)

__call() is triggered when invoking inaccessible methods in an object context.

__callStatic() is triggered when invoking inaccessible methods in a static context.

The *\$name* argument is the name of the method being called. The *\$arguments* argument is an enumerated array containing the parameters passed to the *\$name*'ed method.

Example #2 Overloading methods via the __call() and __callStatic() methods

```
<?php
classMethodTest
{
    public function __call($name, $arguments)
    {
        // Note: value of $name is case sensitive.
        echo "Calling object method '$name' "
            . implode(', ', $arguments). "\n";
    }

    /** As of PHP 5.3.0 */
    public static function __callStatic($name, $arguments)
    {
        // Note: value of $name is case sensitive.
        echo "Calling static method '$name' "
            . implode(', ', $arguments). "\n";
    }
}

$obj = new MethodTest; $obj->runTest('in object context');

MethodTest::runTest('in static context'); // As of PHP 5.3.0
?>
```

The above example will output:

```
Calling object method 'runTest' in object context
Calling static method 'runTest' in static context
```

☺ Object Iteration:

PHP 5 provides a way for objects to be defined so it is possible to iterate through a list of items, with, for example a foreach statement. By default, all visible properties will be used for the iteration.

Example #1 Simple Object Iteration

```
<?php

Class MyClass
{
    public $var1 = 'value 1';
    public $var2 = 'value 2';
    public $var3 = 'value 3';

    protected $protected = 'protected var';
    private $private = 'private var';

    function iterateVisible() {
        echo "MyClass::iterateVisible:\n";
        foreach ($this as $key => $value)
        {
            print "$key => $value\n";
        }
    }
}
```

```

    }
}

$class = new MyClass();

foreach($class as $key => $value)
{
    print "$key => $value\n";
}
echo "\n";

$class->iterateVisible();

?>

```

The above example will output:

```

var1 => value 1
var2 => value 2
var3 => value 3

```

```

MyClass::iterateVisible:
var1 => value 1
var2 => value 2
var3 => value 3
protected => protected var
private => private var

```

As the output shows, the foreach iterated through all of the visible properties that could be accessed.

To take it a step further, the Iterator interface may be implemented. This allows the object to dictate how it will be iterated and what values will be available on each iteration.

Example #2 Object Iteration implementing Iterator

```

<?php
class MyIterator implements Iterator
{
    private $var = array();

    public function __construct($array)
    {
        if (is_array($array)) {
            $this->var = $array;
        }
    }

    public function rewind()
    {
        echo "rewinding\n";
        reset($this->var);
    }

    public function current()
    {
        $var = current($this->var);
        echo "current: $var\n";
        return $var;
    }

    public function key()

```

```

    {
        $var = key($this->var);
        echo "key: $var\n";
        return $var;
    }

    public function next()
    {
        $var = next($this->var);
        echo "next: $var\n";
        return $var;
    }

    public function valid()
    {
        $key = key($this->var);
        $var = ($key !== NULL && $key !== FALSE);
        echo "valid: $var\n";
        return $var;
    }
}
$values = array(1,2,3);
$it = new MyIterator($values);

foreach ($it as $a => $b)
    { print "$a: $b\n";
    }
?>

```

The above example will output:

```

rewinding
valid: 1
current:
1 key: 0
0: 1
next: 2
valid: 1
current:
2 key: 1
1: 2
next: 3
valid: 1
current:
3 key: 2
2: 3
next:
valid
:

```

☺ Magic Method:

The function names __construct(), __destruct(), __call(), __callStatic(), __get(), __set(), __isset(), __unset(), __sleep(), __wakeup(), __toString(), __invoke(), __set state(), __clone() and __debugInfo() are magical in PHP classes. You cannot have functions with these names in any of your classes unless you want the magic functionality associated with them.

Magic Methods in PHP

Magic Method	Description
__construct	This magic methods is called when someone create object of your class. Usually this is used for creating constructor in php5.
__destruct	This magic method is called when object of your class is unset. This is just opposite of __construct.
__get	This method called when your object attempt to read property or variable of the class which is inaccessible or unavailable.
__set	This method called when object of your class attempts to set value of the property which is really inaccessible or unavailable in your class.
__isset	This magic methods trigger when isset() function is applied on any property of the class which is inaccessible or unavailable.
__unset	__unset is something opposite of isset method. This method triggers when unset() function called on inaccessible or unavailable property of the class.
__call	__call magic method trigger when you are attempting to call method or function of the class which is either inaccessible or unavailable.
__callstatic	__callstatic execture when inaccessible or unavailable method is in static context.
__sleep	__sleep methods trigger when you are going to serialize your class object. Syntax: public array __sleep (void)
__wakeup	__wakeup executes when you are un serializing any class object. Syntax: void __wakeup (void)
__toString	__toString executes when you are using echo on your object. Syntax: public string __toString (void)
__invoke	__invoke called when you are using object of your class as function Syntax: mixed __invoke ([\$. . .])
<u>__set_state()</u>	This <u>static</u> method is called for classes exported by <u>var_export()</u> since PHP 5.1.0. The only parameter of this method is an array containing exported properties in the form <i>array('property' => value, ...)</i> . Syntax: static object __set_state (array \$properties)
<u>__debugInfo()</u>	This method is called by <u>var_dump()</u> when dumping an object to get the properties that should be shown. If the method isn't defined on an object, then all public, protected and private properties will be shown. This feature was added in PHP 5.6.0. Syntax: array __debugInfo (void)

Example: __construct() and __destruct

```

<?php
class test
{
    function __construct()
    {
        echo 1;
    }
    function __destruct()
    {
        echo 2;
    }
}

```

```

} $objT = new test();
//__construct get automatically executed and print 1 on screen

unset($objT);
//__destruct triggers and print 2.
?>

```

Example: __get(), __set(), __call(), __callStatic()

```

<?php
class test
{
    function __get($name)
    {
        echo "__get executed with name $name <br>";
    }

    function __set($name , $value)
    {
        echo "__set executed with name $name , value $value <br>";
    }

    function __call($name , $parameter)
    {
        $a = print_r($parameter , true); //taking recursive array in string
        echo "__call executed with name $name , parameter $a <br>";
    }

    static function __callStatic($name , $parameter)
    {
        $a = print_r($parameter , true);
        //taking recursive array in string
        echo "__callStatic executed with name $name , parameter $a <br>";
    }
}

$a = new test();
$a->abc = 3; //__set will executed
$app = $a->pqr; //__get will triggerd
$a->getMyName('ankur' , 'techflirt' , 'etc'); //__call will be executed
test::xyz('1' , 'qpc' , 'test'); //__callstatic will be executed
?>

```

Example : __isset(), __unset()

```

<?php
class test
{
    function __isset($name)
    {
        echo "__isset is called for $name";
    }

    function __unset($name)
    {
        echo "__unset is called for $name";
    }
}

```

```

}
$a = new test();
isset($a->x);
unset($a->c);
?>

```

Example: __sleep(),__wakeup()

```
<?php
```

```
class Connection
```

```
{
```

```
protected $link;
```

```
private $dsn, $username, $password;
```

```
public function __construct($dsn, $username, $password)
```

```
{
```

```
$this->dsn = $dsn;
```

```
$this->username = $username;
```

```
$this->password = $password; $
```

```
this->connect();
```

```
}
```

```
private function connect()
```

```
{
```

```
$this->link = new PDO($this->dsn, $this->username, $this->password);
```

```
}
```

```
public function __sleep()
```

```
{
```

```
return array('dsn', 'username', 'password');
```

```
}
```

```
public function __wakeup()
```

```
{
```

```
$this->connect();
```

```
}
```

```
}?>
```

Example: __toString()

```
<?php
```



```
// Declare a simple class
class TestClass

{

public $foo;

public function __construct($foo)

{

$this->foo = $foo;

}

public function __toString()

{

return $this->foo;

}

}

$class = new TestClass('Hello');
echo $class;

?>
```

Example: __invoke()

```
<?php

class CallableClass

{

public function __invoke($x)

{

var_dump($x);

}

}

$obj = new CallableClass; $obj(5);
var_dump(is_callable($obj)); ?>
```

Example: __set_state()

```
<?php

class A
```

```

{

public $var1;
public $var2;

public static function __set_state($an_array) // As of PHP 5.1.0

{

$obj = new A;

$obj->var1 = $an_array['var1'];
$obj->var2 = $an_array['var2'];

return $obj;
}

}

$a = new A;
$a->var1 = 5;
$a->var2 = 'foo';

eval('$b = ' . var_export($a, true) . '); // $b =

A::__set_state(array( // 'var1' => 5, // 'var2' => 'foo', // ));

var_dump($b);
?>

```

Example: __debugInfo()

```

<?php
class C {
private $prop;

public function __construct($val)
{ $this->prop = $val;
}

public function __debugInfo()
{
return ['propSquared' => $this->prop ** 2,];
}
}

var_dump(new C(42)); ?>

```

😊 Final Keyword:

PHP 5 introduces the final keyword, which prevents child classes from overriding a method by prefixing the definition with *final*. If the class itself is being defined final then it cannot be extended.

In PHP, Final Keyword is applicable to only class and class methods. We cannot declare variables as Final in PHP.

So if you declare class method as a Final then that method **cannot** be override by the child class. Same as method if we declare class as a Final then that class cannot be extended any more.

When to declare a class as final

You should declare a class as final when you think that you implementation of that class should not change in the derived class. You should do this mainly for Utility classes where you don't want the behavior/implementation of your class to change.

Example #1 Final class example

```
<?php

final class BaseClass
{
    public function test()
    {

        echo "BaseClass::test()  called\n";

    }

    // Here it doesn't matter if you specify the function as final or not
    final

    public function moreTesting()
    {

        echo "BaseClass::moreTesting()  called\n";

    }

}

class ChildClass extends BaseClass {

}

// Results in Fatal error: Class ChildClass may not inherit from final
class s (BaseClass)

?>
```

When to declare a method as final

You should declare a class method as final when you think that the method you develop contains necessary functionality to support your application and any modification or change to the functionality can cause unexpected errors/bugs.

Example #2 Final methods example

```
<?php

class BaseClass {

    public function test()
    {

        echo "BaseClass::test()  called\n";

    }

    final public function moreTesting() {
```

```

echo "BaseClass::moreTesting() called\n";
}
}
class ChildClass extends BaseClass
{
public function moreTesting() {

echo "ChildClass::moreTesting() called\n";
}
}
// Results in Fatal error: Cannot override final method
BaseClass::moreTesting()

?>

```

😊 Object Cloning:

if you will directly copy objects in php, then it will copy by reference, not by value. Means if you will change main object data then copied object will be affected.

Also if you will change value of the copied object then main object value will be changed.

So if you want to create copy of the **object which should never referenced to original object then you can take help of object cloning in php.**

As we have seen that typical copy of object in php is by reference. We can copy two object by value in php by value using method of object cloning. Object cloning in php can be implemented using clone keyword.

Creating a copy of an object with fully replicated properties is not always the wanted behavior. A good example of the need for copy constructors, is if you have an object which represents a GTK window and the object holds the resource of this GTK window, when you create a duplicate you might want to create a new window with the same properties and have the new object hold the resource of the new window

An object copy is created by using the *clone* keyword (which calls the object's __clone() method if possible). An object's __clone() method cannot be called directly.

```
$copy_of_object = clone $object;
```

When an object is cloned, PHP 5 will perform a shallow copy of all of the object's properties. Any properties that are references to other variables will remain references.

```
void __clone ( void )
```

Once the cloning is complete, if a __clone() method is defined, then the newly created object's __clone() method will be called, to allow any necessary properties that need to be changed.

Example #1 Cloning an object

```

<?php
class test
{
public $a;

```

```

private $b;
function __construct($a, $b)
{
    $this->a = $a;
    $this->b = $b;
}
}
$a = new test("ankur" , "techflirt");
$b = $a; //Copy of the object
$c = clone $a; //clone of the object
$a->a = "no Ankur";
print_r($a);
print_r($b);
print_r($c);
?>

```

Example #2 Another example of Cloning an object

```

<?php

class SubObject
{

    static $instances = 0;
    public $instance;
    public function __construct()
    {
        $this->instance = ++self::$instances;
    }
    public function __clone()
    {
        $this->instance = ++self::$instances;
    }
}

class MyCloneable
{
    public $object1; public $object2;
    function __clone()

    {
        // Force a copy of this->object, otherwise
        $this->object1 = clone $this->object1;
    }
}

$obj = new MyCloneable();
$obj->object1 = new SubObject();
    $obj->object2 = new SubObject();
$obj2 = clone $obj;
print("Original Object:\n");
    print_r($obj);

```

```
print("Cloned Object:\n");
print_r($obj2);
?>
```

The above example will output:

```
Original Object:
MyCloneable Object
(
    [object1] => SubObject Object
        (
            [instance] => 1
        )
    [object2] => SubObject Object
        (
            [instance] => 2
        )
)
Cloned Object:
MyCloneable Object
(
    [object1] => SubObject Object
        (
            [instance] => 3
        )
    [object2] => SubObject Object
        (
            [instance] => 2
        )
)
```

☺ Comparing Objects:

When using the comparison operator (==), object variables are compared in a simple manner, namely: Two object instances are equal if they have the same attributes and values, and are instances of the same class.

When using the identity operator (===), object variables are identical if and only if they refer to the same instance of the same class.

An example will clarify these rules.

Example #1 Example of object comparison in PHP 5

```
<?php
function bool2str($bool)
{
    if ($bool === false) {
        return 'FALSE';
    } else {
        return 'TRUE';
    }
}

function compareObjects(&$o1, &$o2)
{
    echo 'o1 == o2 : ' . bool2str($o1 == $o2) . "\n";
    echo 'o1 != o2 : ' . bool2str($o1 != $o2) . "\n";
}
```

```

        echo 'o1 === o2 : ' . bool2str($o1 === $o2) . "\n";
        echo 'o1 !== o2 : ' . bool2str($o1 !== $o2) . "\n";
    }

    class Flag
    {
        public $flag;

        function Flag($flag = true) {
            $this->flag = $flag;
        }
    }

    class OtherFlag
    {
        public $flag;

        function OtherFlag($flag = true) {
            $this->flag = $flag;
        }
    }

    $o = new Flag();
    $p = new Flag();
    $q = $o;

    $r = new OtherFlag();

    echo "Two instances of the same class\n";
    compareObjects($o, $p);

    echo "\nTwo references to the same instance\n";
    compareObjects($o, $q);

    echo "\nInstances of two different classes\n";
    compareObjects($o, $r);
?>

```

The above example will output:

```

Two instances of the same class
o1 == o2 : TRUE
o1 != o2 : FALSE
o1 === o2 : FALSE
o1 !== o2 : TRUE

Two references to the same instance
o1 == o2 : TRUE
o1 != o2 : FALSE
o1 === o2 : TRUE
o1 !== o2 : FALSE

Instances of two different classes
o1 == o2 : FALSE
o1 != o2 : TRUE
o1 === o2 : FALSE
o1 !== o2 : TRUE

```

☺ Type Hinting:

TYPE DECLARATIONS

Type declarations were also known as type hints in PHP 5.

Type declarations allow functions to require that parameters are of a certain type at call time. If the given value is of the incorrect type, then an error is generated: in PHP 5, this will be a recoverable fatal error, while PHP 7 will throw a `TypeError` exception.

To specify a type declaration, the type name should be added before the parameter name. The declaration can be made to accept `NULL` values if the default value of the parameter is set to `NULL`.

Valid types

Type	Description	Minimum PHP version
Class/interface name	The parameter must be an <i>instance of</i> the given class or interface name.	PHP 5.0.0
<i>self</i>	The parameter must be an <i>instance of</i> the same class as the one the method is defined on. This can only be used on class and instance methods.	PHP 5.0.0
<u>array</u>	The parameter must be an <u>array</u> .	PHP 5.1.0
<u>callable</u>	The parameter must be a valid <u>callable</u> .	PHP 5.4.0
<u>bool</u>	The parameter must be a <u>boolean</u> value.	PHP 7.0.0
<u>float</u>	The parameter must be a <u>floating</u> point number.	PHP 7.0.0
<u>int</u>	The parameter must be an <u>integer</u> .	PHP 7.0.0
<u>string</u>	The parameter must be a <u>string</u> .	PHP 7.0.0

Aliases for the above scalar types are not supported. Instead, they are treated as class or interface names. For example, using *boolean* as a parameter or return type will require an argument or return value that is an *instance of* the class or interface *boolean*, rather than of type bool:

```
<?php
function test(boolean $param)
{
    test(true);
}>
```


The above example will output:

```
Fatal error: Uncaught TypeError: Argument 1 passed to test() must be an
instance of boolean, boolean given, called in - on line 1 and defined in
- :1
```

Examples

Example: Basic class type declaration

```
<?php
class C
{
}
class D extends C
{
}

// This doesn't extend
C. class E {}

function f(C $c) {
    echo get_class($c)."\n";
}

f(new C);
f(new D);
f(new E);
?>
```

The above example will output:

```
C
D

Fatal error: Uncaught TypeError: Argument 1 passed to f() must be an
instance of C, instance of E given, called in - on line 14 and defined in
- :8
Stack trace:
#0 -(14): f(Object(E))
#1 {main}
    thrown in - on line 8
```

Example: Basic interface type declaration

```
<?php
interface I { public function f(); }
class C implements I { public function f() {} }

// This doesn't implement
I. class E {}

function f(I $i) {
    echo get_class($i)."\n";
}

f(new C);
f(new E);
?>
```

The above example will output:

C

```
Fatal error: Uncaught TypeError: Argument 1 passed to f() must implement
interface I, instance of E given, called in - on line 13 and defined in -
:8
Stack trace:
#0 - (13): f(Object(E))
#1 {main}
  thrown in - on line 8
```

Strict typing

By default, PHP will coerce values of the wrong type into the expected scalar type if possible. For example, a function that is given an integer for a parameter that expects a string will get a variable of type string.

It is possible to enable strict mode on a per-file basis. In strict mode, only a variable of exact type of the type declaration will be accepted, or a TypeError will be thrown. The only exception to this rule is that an integer may be given to a function expecting a float. Function calls from within internal functions will not be affected by the *strict_types* declaration.

To enable strict mode, the *declare* statement is used with the *strict_types* declaration:

Caution

Enabling strict mode will also affect return type declarations.

Note:

Strict typing applies to function calls made from *within* the file with strict typing enabled, not to the functions declared within that file.

Strict typing is only defined for scalar type declarations, and as such, requires PHP 7.0.0 or later, as scalar type declarations were added in that version.

Example: Strict typing

```
<?php
declare(strict_types=1)
;

function sum(int $a, int $b)
{ return $a + $b;
}

var_dump(sum(1, 2));
var_dump(sum(1.5, 2.5));
?>
```

The above example will output:

```
int(3)

Fatal error: Uncaught TypeError: Argument 1 passed to sum() must be of
the type integer, float given, called in - on line 9 and defined in -:4
Stack trace:
#0 - (9): sum(1.5,
2.5) #1 {main}
  thrown in - on line 4
```

Example Weak typing

```
<?php
function sum(int $a, int $b)
    { return $a + $b;
    }

var_dump(sum(1, 2));

// These will be coerced to integers: note the output
below! var_dump(sum(1.5, 2.5));
?>
```

The above example will output:

```
int(3)
int(3)
```

Example: Catching TypeError

```
<?php
declare(strict_types=1)
;

function sum(int $a, int $b)
    { return $a + $b;
    }

try {
    var_dump(sum(1, 2));
    var_dump(sum(1.5, 2.5));
} catch (TypeError $e) {
    echo 'Error: ' . $e->getMessage();
}
?>
```

The above example will output:

```
int(3)
Error: Argument 1 passed to sum() must be of the type integer, float
given, called in - on line 10
```

☺ Late Static Bindings:

As of PHP 5.3.0, PHP implements a feature called late static bindings which can be used to reference the called class in a context of static inheritance.

More precisely, late static bindings work by storing the class named in the last "non-forwarding call". In case of static method calls, this is the class explicitly named (usually the one on the left of the `::` operator); in case of non static method calls, it is the class of the object. A "forwarding call" is a static one that is introduced by `self::`, `parent::`, `static::`, or, if going up in the class hierarchy, forward static call(). The function get_called_class() can be used to retrieve a string with the name of the called class and `static::` introduces its scope.

This feature was named "late static bindings" with an internal perspective in mind. "Late binding" comes from the fact that `static::` will not be resolved using the class where the

method is defined but it will rather be computed using runtime information. It was also called a "static binding" as it can be used for (but is not limited to) static method calls.

Limitations of *self::*:

Static references to the current class like *self::* or *__CLASS__* are resolved using the class in which the function belongs, as in where it was defined:

Example #1 *self::* usage

```
<?php
class A {
    public static function who()
    { echo __CLASS__;
    }
    public static function test()
    { self::who();
    }
}

class B extends A {
    public static function who()
    { echo __CLASS__;
    }
}

B::test();
?>
```

The above example will output:

A

LATE STATIC BINDINGS' USAGE

Late static bindings tries to solve that limitation by introducing a keyword that references the class that was initially called at runtime. Basically, a keyword that would allow you to reference *B* from *test()* in the previous example. It was decided not to introduce a new keyword but rather use *static* that was already reserved.

Example #2 *static::* simple usage

```
<?php
class A {
    public static function who()
    { echo __CLASS__;
    }
    public static function test() {
        static::who(); // Here comes Late Static Bindings
    }
}

class B extends A {
    public static function who()
    { echo __CLASS__;
    }
}

B::test();
?>
```

The above example will output:

B

Note:

In non-static contexts, the called class will be the class of the object instance. Since *\$this->* will try to call private methods from the same scope, using *static::* may give different results. Another difference is that *static::* can only refer to static properties.

Example #3 *static::* usage in a non-static context

```
<?php
class A {
    private function foo() {
        echo "success!\n";
    }
    public function test()
    { $this->foo();
      static::foo();
    }
}

class B extends A {
    /* foo() will be copied to B, hence its scope will still be A
       and * the call be successful */
}

class C extends A {
    private function foo() {
        /* original method is replaced; the scope of the new one is C */
    }
}

$b = new B();
$b->test();
$c = new C();
$c->test();    //fails
?>
```

The above example will output:

```
success!
success!
success!
```

```
Fatal error: Call to private method C::foo() from context 'A' in
/tmp/test.php on line 9
```

😊 Objects and References:

One of the key-points of PHP 5 OOP that is often mentioned is that "objects are passed by references by default". This is not completely true. This section rectifies that general thought using some examples.

A PHP reference is an alias, which allows two different variables to write to the same value. As of PHP 5, an object variable doesn't contain the object itself as value anymore. It only contains an object identifier which allows object accessors to find the actual object. When an object is sent by argument, returned or assigned to another variable, the different variables are not aliases: they hold a copy of the identifier, which points to the same object.

Example #1 References and Objects

```
<?php
class A {
    public $foo = 1;
}

$a = new A;
$b = $a;      // $a and $b are copies of the same identifier
              // ($a) = ($b) = <id>

$b->foo = 2;
echo $a->foo."\n";

$c = new A;
$d = &$c;     // $c and $d are references
              // ($c,$d) = <id>

$d->foo = 2;
echo $c->foo."\n";

$e = new A;

function foo($obj) {
    // ($obj) = ($e) = <id>
    $obj->foo = 2;
}

foo($e);
echo $e->foo."\n";

?>
```

The above example will output:

```
2
2
2
```